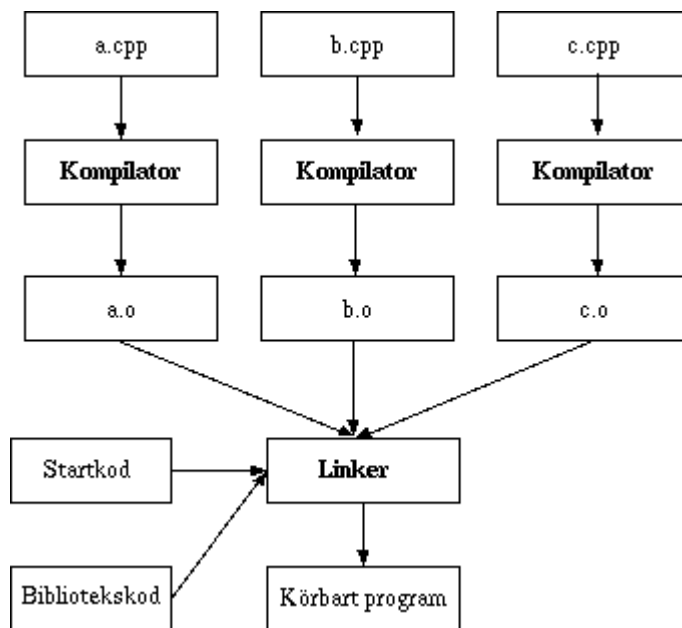


Använda multipla filer

De flesta program når förr eller senare en kritisk storlek då det inte längre är överskådligt att placera all kod i en och samma fil. Var denna gräns kommer är beroende på programmeraren, men endel personer tycker om att ha många relativt små filer, medan andra föredrar större men färre filer. I C++ finns det förstås möjligheter till detta.

Då man har flera filer som skall i sista hand bli ett och samma program bör de enskilda filerna kompileras skilt för sig till s.k. *objektfiler*, som innehåller koden i assemblerform. När alla filer kompilerats till objektfiler länkas de alla ihop tillsammans med nödvändiga bibliotek och ett körbart program skapas. Exemplet nedan visar schematiskt hur ett program bestående av filerna `a.cpp`, `b.cpp` och `c.cpp` för kompileras till motsvarande objektfiler (`.o`) och sedan länkas till ett körbart program.

Figur 13-2. Kompilatorns arbetsskeden vid multipla filer



För att åstadkomma detta måste man se till att kompilatorn inte direkt försöker länka en fil som kompileras, utan denna skall "lämnas" i objektformatet. Man använder då flaggan `-c`. Man bör då inte ge med flaggor som har att göra med t.ex. bibliotek (`-L` och `-l`) men nog flaggor som har att göra med t.ex. var headerfiler finns. När alla filer kompierats är det dags att länka ihop dem till ett enda exekverbart program. Då ger man alla objektfiler på kommandoraden till kompilatorn och inkluderar eventuella länkningsflaggor, varvid kompilatorn skapar programmet (förutsatt att allt är korrekt). Ovanstående program kunde kompileras på följande sätt:

```
% g++ -c a.cpp
% g++ -c b.cpp
% g++ -c c.cpp
% ls
a.cpp a.o b.cpp b.o c.cpp c.o
% g++ a.o b.o c.o -o MittProgram
%
```

Exempel på separat kompilering

Som ett exempel på hur man kan använda separat kompilering och headerfiler i ett eget projekt skall vi titta på ett exempelprogram som definierar en enkelt *stack* (se [avsnittet Allokering av sammansatta datatyper i Kapitel 12](#)). Denna stack är simpel och innehåller endast några få funktioner. Vi skall separera implementationen av stacken i en skild fil `Stack.cpp` och definitionen av funktionsprototyper för stacken i `Stack.h`. Sedan skapar vi ett enkelt huvudprogram i en tredje fil `StackMain.cpp` som använder sig av vår stack.

Definitionsfilen `Stack.h` ser t.ex. ut på följande sätt:

```
// Stack.h

// se till att vi inte får problem om filen inkluderas multipla gånger
#ifndef STACK_H
#define STACK_H

// inkludera headers som kan behövas
#include <iostream>

// definiera ett element i stacken
struct Element {
    // data för detta element
    char Data;
    // nästa element under detta i stacken
    Element * Previous;
};

// funktionsprototyper
Element * push (Element * Top);
Element * pop (Element * Top, char & Data);
bool isEmpty (Element * Top);

#endif
```

Filen `Stack.cpp` som innehåller den egentliga koden för alla funktioner ser ut som nedan. Den måste även inkludera `Stack.h` för att `Element` skall vara definierad.

```
// Stack.cpp -- implementation av stacken

// inkludera våra egna definitioner
#include "Stack.h"

Element * push (Element * Top) {
    char Data;

    cout << "Elementets värde: ";
    cin >> Data;

    // skapa nytt element
    Element * New = new Element;
    New->Data = Data;
    New->Previous = Top;

    // återvänd och returnera nya topp-elementet
    return New;
}

// pop:a ett element från stacken
Element * pop (Element * Top, char & Data) {
```

```

// är stacken tom?
if ( Top == 0 ) {
    // stacken tom
    return 0;
}

// spara data som är lagret i elementet
Data = Top->Data;

// spara pekare till det element som blir nytt topp-element
Element * Tmp = Top->Previous;

// frigör minne och återvänd
delete Top;
return Tmp;
}

// kolla om stacken är tom
bool isEmpty (Element * Top) {

    if ( Top == 0 ) {
        // stacken tom
        return true;
    }

    // stacken inte tom
    return false;
}

```

Själva huvudprogrammet som använder sig av vår stack ser ut som nedan. Den enda skillnaden mellan detta program och det i [avsnittet Allokering av sammansatta datatyper i Kapitel 12](#) är att nu finns hela stacken definierad och implementerad i andra filer, och det enda som behövs är en `#include "Stack.h"`.

```

#include "Stack.h"

int main () {
    Element * Top = 0;
    char Choice = ' ';
    char Data;

    // iterera tills användaren vill avsluta
    while ( Choice != '3' ) {
        cout << "Välj: " << endl << "1 - push" << endl << "2 - pop" << endl;
        cout << "3 - sluta " << endl << "-> ";

        // läs in ett menyval
        cin >> Choice;

        // vad vill användaren göra
        switch ( Choice ) {
            case '1' : Top = push ( Top ); break;
            case '2' :
                // har vi element i listan?
                if ( ! isEmpty ( Top ) ) {
                    // ja, poppa och skriv ut det poppade värdet
                    Top = pop ( Top, Data );
                    cout << "Poppade: " << Data << endl;
                }
                else {
                    cout << "Stacken tom!" << endl;
                }
            }
        }
    }
}

```

```
        break;
    }
}
```

Programmet kan sedan kompileras med följande kommandon:

```
% g++ -c Stack.cpp
% g++ StackMain.cpp Stack.o
% ./Stack
...
```

Notera att vi här kompilerade och länkade `StackMain.cpp` på samma gång genom att räkna upp de objekt-filer vi ville länka med på kommandoraden. Vi kunde även kompilerat även denna fil men flaggan `-c` och sedan gjort en separat länkning av programmet.

[Förutgående](#)

Länkning av program

[Hem](#)
[Upp](#)

[Nästa](#)
Klasser